

13 E Holes (2700 Rated)

Authored by: Arpit Raj, LNMIIT Jaipur, August 14, 2026

Problem Understanding & Rules

- holes numbered 1 to N
- hole i has power a_i
- throw into i , jump to $[i + a_i]$
- M moves
- If 0 then set power of hole a to b
- If 1 then throw a ball to a and count number of jumps of a ball before come out and write number of hole it come from

O/p for each type 1

- number of holes the ball visited $\rightarrow x$
- last hole it visited $\rightarrow z$

I/P

- i) N, M
- ii) initial powers
- $\rightarrow$ then following M lines contains moves either of type:
 - 0 a b
 - 1 a

Initial Approach (TLE)

So basically $\rightarrow$ TLE de raha

```
while (i <= n) {
    i = i + a[i];
    jumps++;
}
```

ok for one query but there are 10^5 queries.

Square Root Decomposition Idea

for every i , I need:

- $\text{next}[i]$ → where we end up after jumping several times until leave current block
- $\text{cnt}[i]$ → how many jumps those jumps took
- $\text{last}[i]$ → last hole visited before leaving the block

Divide the Array into Blocks

Eg $n = 16$, $\sqrt{n} \approx 4$

```
[ 0 1 2 3 ]  
[ 4 5 6 7 ]  
[ 8 9 10 11 ]  
[12 13 14 15 ]
```

now if in future $a[2] = \text{new}$, then only the block with index 2 is affected.

So:

- $\text{next}[i]$ → first index reached outside curr block
- $\text{cnt}[i]$ → jumps needed to get there
- last → before leaving entire array, where was the ball

Building Blocks

let $B = \sqrt{n} + 1$

$a = [1\ 1\ 1\ 1\ | \ 2\ 1\ 3\ 1\ | \ \dots]$

let $i = 1$

normal sim → $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

4 is outside block $[0..3]$

```
next = i + a[i]
```

- If next leaves array:

- `nxt[i] = next`
- `cnt[i] = 1`

- If next in some block:

- `nxt[i] = nxt[next]`
- `cnt[i] = cnt[next] + 1;`

- that's why process R to L

Build Function Logic

```
void build(int block) {
    int l = block * B;
    int r = min(n - 1, (block + 1) * B - 1);
    for (int i = r; i >= l; i--) {
        int next = i + a[i];
        if (next >= n) { // left array
            nxt[i] = next;
            cnt[i] = 1;
        }
        else if (next / B != i / B) { // next block
            nxt[i] = next;
            cnt[i] = 1;
        }
        else {
            nxt[i] = nxt[next];
            cnt[i] = cnt[next] + 1;
        }
    }
}
```

Query Updates

now 0 a b type

```
arr[a] = b
```

`build(a/B) → O( $\sqrt{n}$ )`

only block containing a becomes invalid and hence we rebuild it.

Final Source Code

```
#include <bits/stdc++.h>
using namespace std;

vector<int> arr;
int n, m;
int B;

vector<int> nxt;
vector<int> cnt;
vector<int> lasthole;

void build(int block) {
    int l = block * B;
    int r = min(n - 1, (block + 1) * B - 1);

    for (int i = r; i >= l; i--) {
        int next = i + arr[i];
        if (next >= n) {
            nxt[i] = next;
            cnt[i] = 1;
            lasthole[i] = i;
        } else if (next / B != i / B) {
            nxt[i] = next;
            cnt[i] = 1;
            lasthole[i] = i;
        } else {
            nxt[i] = nxt[next];
            cnt[i] = cnt[next] + 1;
            lasthole[i] = lasthole[next];
        }
    }
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cin >> n >> m;

    arr.resize(n);
    nxt.resize(n);
    cnt.resize(n);
    lasthole.resize(n);

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    B = sqrt(n) + 1;
    int blocks = (n + B - 1) / B;

    for (int i = 0; i < blocks; i++) {
        build(i);
    }
}
```

```

}

while (m--) {
    int type;
    cin >> type;

    if (type == 0) {
        int pos, val;
        cin >> pos;
        pos--;
        cin >> val;

        arr[pos] = val;
        build(pos / B);
    } else {
        int pos;
        cin >> pos;

        pos--;

        int last = pos;
        int jmp = 0;

        while (pos < n) {
            last = lasthole[pos];
            jmp += cnt[pos];
            pos = nxt[pos];
        }

        cout << last + 1 << " " << jmp << "\n";
    }
}

return 0;
}

```